

GUI — ВЕБ-ПУЛЬТ РОБОТА-КУРЬЕРА (ДИСПЕТЧЕР АВТОНОМНОЙ ЛОГИСТИКИ)

Веб-пульт робота-курьера НТЦ АО «АВТОВАЗ» (проект РУС-SLAM, шасси 4WIS/4WID). Исходный код: [gui/](#). Дизайн-язык перенесён из [ZMK Vision Console](#) (токены, темы, панели, сегментные вкладки, LIVE/REC-бейджи) и адаптирован под раскладку диспетчерского макета: тёмная тема, циановый акцент.

Стек: **React 19 + TypeScript + Vite 6**. Никаких серверных зависимостей: это статическое SPA, которому достаточно любого компьютера с браузером — пульт одинаково запускается на **Ubuntu/Linux** и **Windows**.

Документ описывает: быстрый запуск на обеих ОС, раскладку экрана, все панели, горячие клавиши, кастомизацию, демо-симулятор, структуру исходного кода, прод-сборку и план перевода на живые данные ROS 2.

1. Быстрый запуск

1.1. Требования

Компонент	Версия	Проверка
Node.js	18+ (рекомендуется LTS 20/22)	<code>node -v</code>
npm	поставляется с Node.js	<code>npm -v</code>

Ничего больше не требуется: зависимости поставит сам скрипт запуска.

1.2. Запуск на Ubuntu / Linux

В репозитории есть скрипт [gui/start.sh](#):

```
cd gui
chmod +x start.sh          # один раз
./start.sh                 # режим разработки → http://localhost:5173
./start.sh --prod          # прод-сборка и просмотр → http://localhost:4173
./start.sh --port 8080     # другой порт (работает с обоими режимами)
./start.sh --help          # справка
```

Скрипт сам проверяет наличие и версию Node.js, при первом запуске выполняет `npm install`, затем поднимает сервер. Если Node.js не установлен:

```
sudo apt update && sudo apt install -y nodejs npm
# либо свежий LTS через NodeSource:
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs
```

Ручной вариант без скрипта:

```
cd gui
npm install
npm run dev          # http://localhost:5173 (bind 0.0.0.0)
```

1.3. Запуск на Windows

В репозитории есть скрипт [gui/start.bat](#) — работает и двойным кликом из Проводника, и из cmd:

```
cd gui
start.bat          — режим разработки → http://localhost:5173
start.bat --prod   — прод-сборка и просмотр → порт 4173
start.bat --port 8080 — другой порт
start.bat --help   — справка
```

Если Node.js не установлен — скрипт подскажет; установка:

```
winget install OpenJS.NodeJS.LTS
```

или скачайте LTS-инсталлятор с и перезапустите терминал. Ручной вариант из cmd/PowerShell полностью совпадает с линуксовым: `npm install`, затем `npm run dev`.

Скрипт привязан к каталогу, в котором лежит (`cd /d "%~dp0"` в `.bat`, `cd "$(dirname "$0")"` в `.sh`), поэтому его можно копировать и вызывать из любого места — зависимости и конфиги всегда находятся правильно.

1.4. Что откроется

После запуска откройте в браузере **http://localhost:5173** (или указанный `--port`). Через ~2 с после старта демо-симулятор сам отправит задание «КТП → Склад», и робот поедет по карте — можно сразу смотреть, как всё работает, без робота и ROS 2.

2. Раскладка диспетчерского экрана

НТЦ АВТОВАЗ		[КАРТА КАМЕРА]	Демо • 95% • 0.0 км/ч • LiDAR
ПАНЕЛЬ ЗАКАЗОВ А: КТП [↔] В: Склад [ОТПРАВИТЬ] [E-STOP] СХЕМА ЗАДАНИЯ (блоки миссии)	КАРТА ЛИДАР дороги, здания, зона, маршрут, робот RB-01, метки A/B, pan/zoom		МОНИТОРИНГ И ЗРЕНИЕ (камера CAM-01 + детекции YOLO)
	СХЕМА ЗАДАНИЯ: А→Б → открыть → ждать → заккрыть → ждать → на А → ожидание		РАСПОЗНАВАНИЕ ОБ-ОВ (список с conf) ЖУРНАЛ СОБЫТИЙ (ROSOUT-лог)

Три колонки: слева — управление заданием, в центре — карта лидара или камера на всё окно, справа — мониторинг, распознавание объектов и журнал событий.

3. Элементы интерфейса

3.1. Шапка (Header)

- **Бренд** «НТЦ АВТОВАЗ — диспетчер автономной логистики».
- **Сегментные вкладки «КАРТА | КАМЕРА»** в центре: карта — главное окно; «Камера» разворачивает ленту CAM-01 на весь центр. Переключение — мышью или клавишами 1 / 2.
- **Чипы телеметрии** справа: индикатор режима (Демо), заряд батареи, скорость (км/ч), статус лидара (ЛДС-01), часы, бейджи состояния.

3.2. Панель заказов (левая колонка)

- **Точки А и В** — выбор из четырёх точек территории: КТП, Склад, Администрация, Зона тестирования; кнопка «↔» меняет их местами.
- **«ОТПРАВИТЬ»** — формирует и запускает миссию (код задания вида Т-001).
- **«E-STOP»** — экстренная остановка (также клавиша Space): робот немедленно останавливается, миссия прерывается, событие уходит в журнал.
- **Статус робота и отсека**: фаза автомата миссии (автопатруль, движение к В, открытие/заккрытие отсека, ожидания, возврат на А, ожидание задания), заряд батареи, одометр, состояние отсека (закрыт/открывается/открыт/закрывается).
- **Клик по точке на карте** назначает её пунктом В.

3.3. Схема задания (под панелью заказов)

Миссия представлена как **программа из блоков**: движение $A \rightarrow B \rightarrow$ открыть отсек $\rightarrow$ ожидание $\rightarrow$ закрыть отсек $\rightarrow$ ожидание $\rightarrow$ возврат на $A \rightarrow$ ожидание задания. Активный блок подсвечен и показывает прогресс-бар, пройденные — отметку выполнения. Тайминги действий и ожиданий настраиваются в кастомизации (см. раздел б).

3.4. Карта лидара (центр)

Рендер на <canvas> без внешних библиотек:

- **Пан/зум**: сдвиг — ЛКМ или пинч; зум — колесо мыши к курсору, пинчем или кнопками «+ / — / вписать / следование».
- **Слои**: сетка 10/50 м, дорожная сеть с осевыми линиями, здания (СКЛАД №2, АДМИНИСТРАЦИЯ, КТП), штрихованная зона тестирования, маршрут миссии (активное плечо ярче), метки А и В.
- **Робот RB-01** с лидаром ЛДС-01 в носовой точке (сверху спереди по центру, совпадает с URDF lidar_joint, $x = 0.32$ м): направление, вращающийся луч скана, шлейф траектории, облако точек скана.
- **Служебная строка**: масштабная линейка и координаты $X/Y/\theta$ в шапке.

3.5. Мониторинг и зрение (правая колонка)

- **Камера CAM-01**: в демо-режиме — рейкастинг от позиции робота (вид реагирует на движение и поворот) с bounding-box'ами детекций (STOP sign, Pedestrian, Crosswalk, Traffic light) и уверенностью; при заданном в настройках URL (HLS/WebM) вместо демо-рендера подключается реальный <video>. Бейдж REC, время, FOV 87°.
- **Распознавание объектов (YOLOv8 demo)**: список активных детекций с вероятностью (confidence).
- **Журнал событий (ROSOUT)**: состояния светофора (зелёный/жёлтый/ красный), препятствия, события миссии и E-STOP.

3.6. Кастомизация

Панель открывается кнопкой в шапке или клавишей К. Все настройки сохраняются в localStorage браузера и переживают перезагрузку.

4. Горячие клавиши и мышь

Действие	Клавиши / мышь
Вкладка «Карта»	1
Вкладка «Камера»	2
Открыть/закрыть кастомизацию	К
Экстренная остановка (E-STOP)	Space
Сдвиг карты	ЛКМ (перетаскивание) или пинч
Зум карты	колесо мыши (к курсору), пинч, кнопки «+/-»
Вписать карту / следование за роботом	кнопки панели карты
Назначить пункт В миссии	клик по точке на карте

5. Демо-симулятор

Пульт полностью автономен в демо-режиме (без робота и ROS 2):

- **Тик 10 Гц**; робот **3.5 м/с (≈ 12.6 км/ч)** движется по графу дорог территории (кратчайший путь — алгоритм Дейкстры).
- **Автопатруль** по кольцевому маршруту при отсутствии задания.
- **Полный цикл миссии**: А → Б → открыть отсек → ожидание → закрыть отсек → ожидание → возврат на А → ожидание задания.
- **Телеметрия**: батарея, одометр, скорость, статус фазы.
- **360°-скан лидара ЛДС-01**: лучи castRay по зданиям, препятствиям и границе территории (territory.ts).
- **Детектор объектов** с циклом светофора и детекциями (коробка + уверенность) для правой колонки и оверлея камеры.
- Через **2 с** после запуска автоматически отправляется демо-задание КТП → Склад — интерфейс сразу «оживает».

6. Кастомизация: полный список настроек

Настройка	Варианты	По умолчанию
Тема	светлая / тёмная / авто (системная)	тёмная
Акцент	Циан (фирменный), Лайм (ZMK), Янтарь, Коралл, Фиалка	#38e0e0
Масштаб шрифта	S / M / L	M
Стиль карты	светлый / тёмный	тёмный
Следование за роботом	вкл/выкл	вкл
Сетка / шлейф / скан на карте	вкл/выкл (каждая)	вкл
Масштаб меток	число	1
Качество камеры	низкое (320) / среднее (480) / высокое (640 колонок)	среднее
Оверлеи камеры: сетка, HUD	вкл/выкл	вкл
URL видеопотока	строка (HLS/WebM), пусто — демо-рендер	пусто
Длительность действия с отсек	секунды	3
Ожидание после открытия/закрытия	секунды	3
Сброс настроек	кнопка	—

7. Структура исходного кода

gui/	
index.html	– входная точка (SPA, русская локаль)
vite.config.ts	– dev/preview серверы: 0.0.0.0:5173, без ограничений по origin (удобно для проху и раздачи с мостом)
start.sh	– запуск на Ubuntu/Linux (см. раздел 1.2)
start.bat	– запуск на Windows (см. раздел 1.3)
src/	
main.tsx	– монтирование React-приложения
App.tsx	– состояние пульта, клавиатура, источники данных
styles.css	– дизайн-токены, темы (светлая/тёмная), акценты
lib/	
territory.ts	– территория: точки, дороги, здания, зона, gaucast (здания/препятствия/граница), Дейкстра
simulator.ts	– робот: кинематика, конечный автомат миссии (движение/отсек/ожидания), E-STOP, скан лидара
detect.ts	– демо YOLOv8: цикл светофора, детекции с box/conf
mapRender.ts	– рендер карты (pan/zoom-трансформ, маршрут, робот)
cameraRender.ts	– рейкастинг вида с камеры + оверлей детекций
settings.ts	– настройки + персистентность (localStorage)
logtypes.ts	– типы журнала событий (уровни, время)
components/	
Header.tsx	– шапка: бренд, вкладки Карта/Камера, чипы телеметрии
DispatchPanel.tsx	– панель заказов (A/B, отправка, E-STOP, статус)
SchemePanel.tsx	– схема задания: блоки миссии с прогрессом
MapPanel.tsx	– карта: pan/zoom/pinch, клик по точкам, легенда
CameraFeed.tsx	– лента CAM-01 (демо-рендер или реальный video)
VisionPanel.tsx	– правая колонка: зрение + распознавание + журнал
Customizer.tsx	– панель кастомизации
icons.tsx	– SVG-иконки

8. Прод-сборка и раздача

```
cd gui
npm run build      # typecheck (tsc --noEmit) + vite build → dist/
npm run preview    # локальный просмотр сборки (0.0.0.0:4173)
```

Каталог dist/ — статические файлы: их можно раздавать любым веб-сервером (nginx, Caddy, python3 -m http.server) или отдавать прямо из узла-моста ROS 2 рядом с WebSocket-эндпоинтом. Благодаря host: 0.0.0.0 и allowedHosts: true в vite.config.ts пульт корректно работает и через прокси/превью-серверы.

9. Дальше: мост на живые данные ROS 2

simulator.ts и detect.ts — изолированные источники данных с устойчивыми интерфейсами. Следующий шаг — узел-мост ros2_ws/src/rus_slam_gui_bridge:

Данные	Источник в ROS 2
Поза робота	tf2 (одометрия → map)
Сканы лидара	sensor_msgs/LaserScan (ЛДС-01)
Карта	nav_msgs/OccupancyGrid (slam_toolbox)
Детекции	узел YOLO (rus_slam_perception)
Камера	WebRTC/MJPEG-стрим
Транспорт	WebSocket → браузер

После появления моста GUI переходит с демо-режима на живые данные **без изменения интерфейсов** и раскладки.

10. Частые проблемы

Симптом	Причина / решение
node: command not found (Linux) / «Node.js не найден» (Windows)	Node.js не установлен — см. разделы 1.2/1.3
Ошибка npm install на Windows	перезапустите терминал после установки Node.js; при политиках выполнения используйте cmd, а не PowerShell
Порт 5173 занят	./start.sh --port 8080 или start.bat --port 8080
Страница не открывается с другого компьютера	сервер слушает 0.0.0.0 — проверьте фаервол (sudo ufw allow 5173)
Настройки «забылись»	настройки живут в localStorage конкретного браузера/профиля
Видео по URL не играет	поддерживаются HLS/WebM-потoki, доступные браузеру напрямую

*Документ соответствует ревизии GUI **v0.2** (демо-симулятор, схема задания, кастомизация).
Исходники: [gui/](#), краткая справка — [gui/README.md](#).*